

Practical: 2

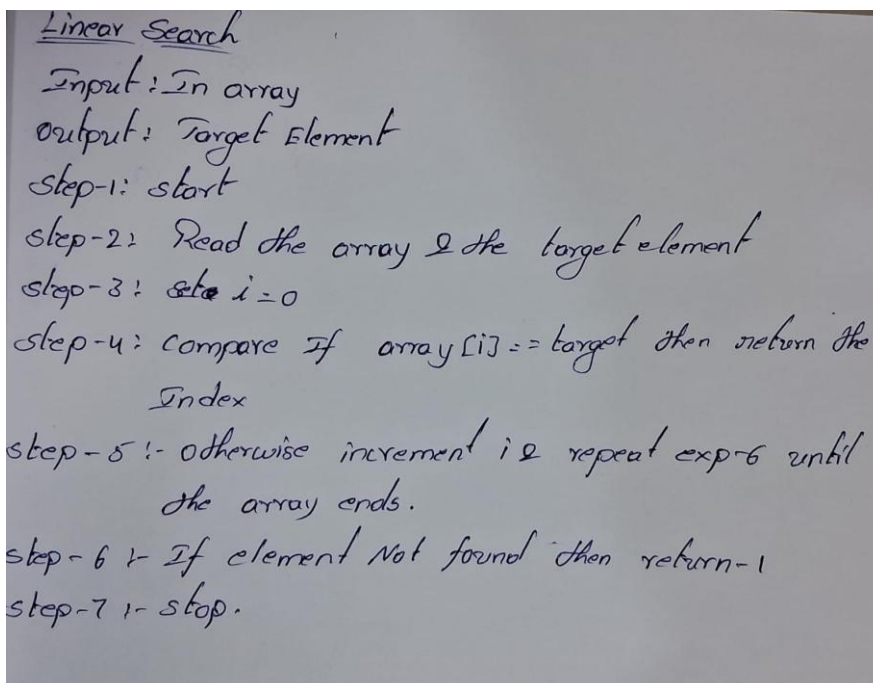
Implementation and Time analysis of linear and binary search algorithm

05/08/2026

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Linear search

Algorithm:



Linear Search
Input: In array
Output: Target Element
Step-1: start
Step-2: Read the array & the target element
Step-3: set $i = 0$
Step-4: compare If $\text{array}[i] == \text{target}$ then return the Index
Step-5: otherwise increment i & repeat exp-6 until the array ends.
Step-6: If element Not found then return -1
Step-7: stop.

Program:

```
#include <iostream>

using namespace std;

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];    cout << "Enter " << n <<
" elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    int target;    cout << "Enter the element
to search: ";
    cin >> target;

    int index = -1;    for (int i
= 0; i < n; i++) {        if
(arr[i] == target) {
        index = i;        break;
    }
}

    if (index != -1) {        cout << "Element found at index:
" << index << endl;
    } else {
        cout << "Element not found" << endl;
    }

    return 0;
}
```

Output:

```
Enter the number of elements: 5
Enter 5 elements: 66
32
55
45
6
Enter the element to search: 6
Element found at index: 4
```

Time Complexity:

Time Complexity

```
for(i=0; i<n; i++) { n+1
    if(arr[i] == t) { -n
        return i; -> 1
    }
}
return -1 -> 1
}
```

$2n + 2 + n + 1 + 1$
 $3n + 4$
 $T(n) = O(n)$

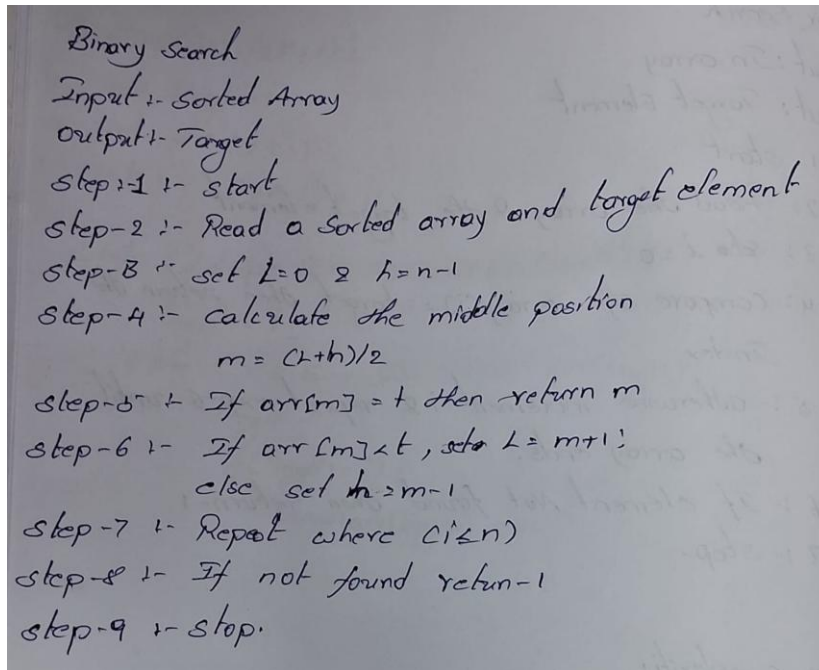
Bestcase:-
 $arr[5] = \{1, 2, 3, 4, 5\}$
 $t(n) = 1$
 $T(n) = O(1)$

Average Case
 $arr[5] = \{1, 2, 3, 4, 5\}$
 $key = 3 \quad n/2$
 $T(n) = n/2$
 $T(n) = O(n)$

Worstcase:- $arr[5] = \{1, 2, 3, 4, 5\}$
 $T(n) = O(n)$

Binary search

Algorithm:



Binary Search
Input :- Sorted Array
Output :- Target
Step-1 :- start
Step-2 :- Read a sorted array and target element
Step-3 :- set $l=0$ & $h=n-1$
Step-4 :- calculate the middle position
 $m = (l+h)/2$
Step-5 :- If $arr[m] = t$ then return m
Step-6 :- If $arr[m] < t$, set $l = m+1$;
else set $h = m-1$
Step-7 :- Repeat where $l \leq n$
Step-8 :- If not found return -1
Step-9 :- stop.

Program code:

```
#include <iostream>
#include <vector>

using namespace std;

int binarySearch(const vector<int>& arr, int target) {
    int low = 0;    int high = arr.size() - 1;

    while (low <= high) {        int
        mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            low = mid + 1;        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int main()
```

```
{
    int size;    cout << "Enter the number of
elements: ";
    cin >> size;

    vector<int> arr(size);    cout << "Enter " << size << "
elements in sorted order:\n";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }

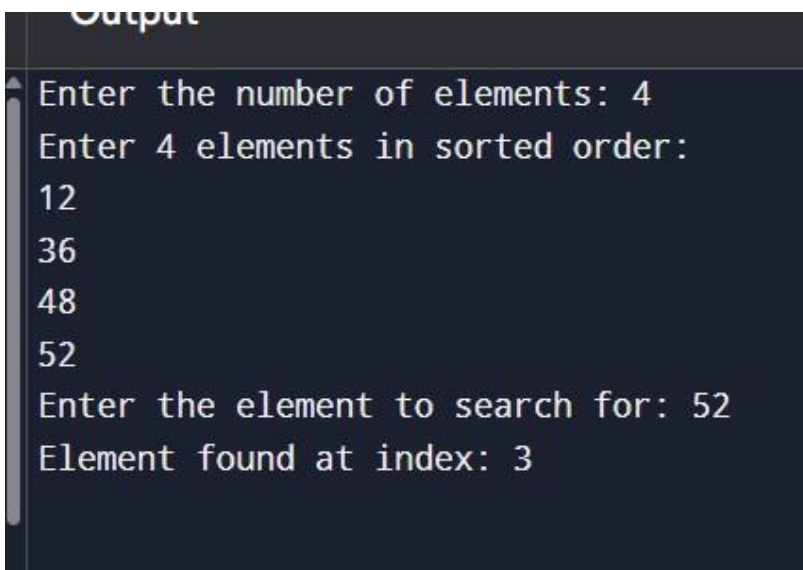
    int target;    cout << "Enter the element to
search for: ";
    cin >> target;

    int result = binarySearch(arr, target);

    if (result != -1) {        cout << "Element found at index:
" << result << endl;
    } else {
        cout << "Element not found in the array." << endl;
    }

    return 0;
}
```

Output:



```
Output
Enter the number of elements: 4
Enter 4 elements in sorted order:
12
36
48
52
Enter the element to search for: 52
Element found at index: 3
```

+

Time complexity:

Handwritten notes on binary search algorithm and its time complexity analysis.

```

int binarySearch(int arr[], s, e) {
    l = 0;
    h = s - 1;
    while (l <= h) {
        m = (l + h) / 2;
        if (arr[m] == x) {
            return m;
        }
        else if (arr[m] < x) {
            l = m + 1;
        }
    }
}

```

Time complexity analysis:

- Best case: $\text{mid} = \text{low/high} = 1$
 $\text{if (arr[mid] == key) - 1}$
 $T(n) = 1 + 1 = 2$
 $T(n) = O(1)$
- Average case: $O(\log n)$
 $n \rightarrow \frac{n}{2} + \frac{n}{4} + \dots$
 $T(n) = O(\log n)$
- Worst case: $O(\log n)$
 $n \rightarrow \frac{n}{2} + \frac{n}{4} + \dots$
 $T(n) = \log_2 n$
 $= O(\log n)$

Additional notes on the right side of the page:

- $m = \frac{n}{2} + \frac{n}{2}^2 + \frac{n}{2}^3$
- $\left\{ \frac{n}{2} \right\}^k$
- $\frac{n}{2^k} = 1$
- $n = 2^k$
- $k = \log n$
- Time complexity: $T(n) = O(\log n)$

Conclusion:

Hence, from this practical, the Linear Search and Binary Search algorithms and programs have been executed successfully. Their time complexities have been analyzed and categorized into Best, Average, and Worst cases.